

INFO-F-524 Continuous Optimization

Project report

Silberwasser Elliot 518397 Master I, Aljendi Lyan 492045 Master II

May 30, 2025



Contents

1	Abstract	2
2	Introduction	2
2.1	Regularized regression models	2
2.1.1	Linear Regression – Normal Equation	2
2.1.2	l_1 -regularization - LASSO	3
2.1.3	l_2 -regularization - Ridge	3
2.1.4	$l_1 - l_2$ -regularization - Elastic Net	4
2.2	Gradient Descent Method (GDM)	4
2.3	Proximal Gradient Method (PGM)	6
2.4	Specifications of the ISTA algorithm	7
2.5	Specifications of the FISTA algorithm	7
3	Design	8
3.1	Programming language motivations	8
3.2	The Iterative Solver	8
3.3	The regularized regression models	11
4	Results	13
4.1	Comparison of the results	15
4.1.1	Which is the best search step size method ?	16
4.1.2	Which is the best iterative algorithm ?	16
4.1.3	Which regularized regression model best fits our dataset ?	16
5	Conclusion	16
6	Project GitHub repository	17

1 Abstract

This project focuses on the implementation and comparison of continuous optimization algorithms applied to regression problems over datasets. In particular, we study regularized regression models using the Mean Squared Error (MSE) loss combined with different regularization terms: l_1 -norm (Lasso), l_2 -norm (Ridge), and the combined l_1 - l_2 -norm (Elastic Net).

We implement four iterative optimization algorithms specifically designed to handle such composite objective functions: the Gradient Descent Method, the Proximal Gradient Method, the Iterative Shrinkage-Thresholding Algorithm (ISTA) and its accelerated variant, FISTA. This report presents the mathematical background and algorithmic details underlying those methods. Finally, we evaluate and compare their performance on a dataset using different regularization schemes, highlighting their convergence behavior and effectiveness.

2 Introduction

To understand exactly the difference between the algorithms, we present here the mathematical background of their operation.

2.1 Regularized regression models

In this subsection, we recall the principles behind several regression models: the normal equation, l_1 -regularization (Lasso), l_2 -regularization (Ridge), and combined $l_1 - l_2$ -regularization (Elastic Net). In fact, to test our algorithms, it was asked to do so with datasets and by carrying out these different regulations on them.

2.1.1 Linear Regression – Normal Equation

We consider a specific objective function known as the Mean Squared Error (MSE) loss. This function arises from a given dataset composed of:

- A response vector $y = (y_1, \dots, y_m)^\top \in \mathbb{R}^m$, containing m observed values of the dependent variable;
- A model matrix $X = (x_1 \mid \dots \mid x_n) \in \mathbb{R}^{m \times n}$, where each column $x_j = (x_{1j}, \dots, x_{mj})^\top$ represents one of the n predictor variables.

The regression model assumes a linear relationship between the predictors and the response of the following form:

$$y = \beta_0 + x_1\beta_1 + \dots + x_n\beta_n,$$

which can be rewritten in matrix form as:

$$y = X\beta,$$

where $\beta = (\beta_0, \beta_1, \dots, \beta_n)^\top \in \mathbb{R}^{n+1}$ is the vector of regression coefficients.

To fit this linear model to the data, we aim to find the vector β that minimizes the residual error $y - X\beta$, typically by minimizing the squared Euclidean norm:

$$\|y - X\beta\|^2.$$

This leads to a convex quadratic objective function, which ensures that optimization algorithms can efficiently converge to the global minimum. Therefore, due to the strict convexity

of the objective function, any local minimum is unique and also corresponds to the global minimum.

In particular, the minimizer of the least squares loss function satisfies the called normal equation:

$$X^\top(y - X\hat{\beta}) = 0.$$

This equation is obtained by setting the gradient of the objective function to zero. Since the function is convex, solving this equation guarantees convergence to the global minimizer.

The final objective MSE function is defined as:

$$\min_{\beta \in \mathbb{R}^n} \left\{ \frac{1}{2n} \|y - X\beta\|_2^2 \right\} \quad (1)$$

2.1.2 l_1 -regularization - LASSO

The Least Absolute Shrinkage and Selection Operator (LASSO) is a linear regression method that introduces l_1 -norm regularization to enforce sparsity in the solution. While the standard linear regression model seeks to minimize the squared error $\|y - X\beta\|^2$, LASSO modifies the objective function by adding a penalty on the absolute values of the coefficients:

$$\min_{\beta \in \mathbb{R}^n} \left\{ \frac{1}{2n} \|y - X\beta\|^2 + \lambda \|\beta\|_1 \right\},$$

where $\|\beta\|_1 = \sum_{j=1}^n |\beta_j|$ denotes the l_1 -norm of β , and $\lambda > 0$ is a regularization parameter that controls the trade-off between data fitting and model sparsity. In optimization, sparsity means that the solution has many zero coefficients. This leads to simpler, more interpretable, and more efficient model because it uses fewer variables.

Unlike the standard least squares problem, the LASSO objective is not differentiable due to the absolute value function in the regularization term. Therefore, classical closed-form solutions such as the normal equation cannot be applied. A common approach is the Proximal Gradient Method, where the non-differentiable l_1 -term is handled via the soft-thresholding operator.

2.1.3 l_2 -regularization - Ridge

Ridge regression is a linear regression method that introduces l_2 -norm regularization in order to shrink the coefficients and prevent overfitting. Like LASSO, Ridge starts from the standard least squares formulation but adds a penalty on the squared magnitude of the regression parameters β :

$$\min_{\beta \in \mathbb{R}^n} \left\{ \frac{1}{2n} \|y - X\beta\|^2 + \frac{\lambda}{2} \|\beta\|_2^2 \right\},$$

where $\|\beta\|_2^2 = \sum_{j=1}^n \beta_j^2$ denotes the squared l_2 -norm of the parameters vector, and $\lambda > 0$ is a regularization parameter controlling the strength of the penalty.

A major advantage of Ridge regression is that the objective function remains strictly convex and differentiable (due to the sum of strictly convex functions). This allows for a closed-form solution similar to the normal equation. A closed-form solution refers to an explicit mathematical expression that gives the exact solution to an optimization problem without the need for iterative computation.

However, although closed-form solutions are available, they become impractical for large-scale datasets. In that case, iterative methods offer a scalable alternative, avoiding expensive computation cost while still converging to the optimal solution.

2.1.4 $l_1 - l_2$ -regularization - Elastic Net

Elastic Net is a regularized regression model that combines both l_1 -norm (Lasso) and l_2 -norm (Ridge) penalties in a single objective function. It was introduced to overcome the limitations of Lasso, particularly when dealing with highly correlated predictors or when the number of predictors exceeds the number of observations. The Elastic Net objective function is defined as:

$$\min_{\beta \in \mathbb{R}^n} \left\{ \frac{1}{2n} \|y - X\beta\|^2 + \lambda_1 \|\beta\|_1 + \frac{\lambda_2}{2} \|\beta\|_2^2 \right\},$$

where $\lambda_1 > 0$ and $\lambda_2 > 0$ are regularization parameters controlling the contributions of the l_1 and l_2 terms, respectively.

Due to the non-differentiability of the l_1 -term, Elastic Net does not admit a closed-form solution and must be solved using iterative optimization methods. Proximal algorithms, such as ISTA or FISTA.

2.2 Gradient Descent Method (GDM)

This algorithm is the underlying iterative process used to implement the other algorithms, notably PGM, ISTA and FISTA.

The principle behind the **GDM** is to minimize a given objective function. In our case, we want to perform a linear regression as described in the previous subsection and find the coefficients that minimize the mean-square error between the predicted and actual values of a given dataset. We implement the variant called **Gradient Descent Method with backtracking line search**.

This algorithm makes several assumptions about the function it is processing.

1. The objective function f is convex and differentiable with $\text{dom}(f) = \mathbb{R}^n$. This implies the First-order condition for convexity described as follows:

$$f(y) \geq f(x) + \nabla f(x)^T (y - x), \forall x, y \in \text{dom}(f)$$

In other terms, it provides a lower bound. In particular, it gives a lower bound (approximation from below) on $f(y)$. Due to the convexity of the function, this inequality reflects the fact that the tangent is always below the graph of the function.

2. ∇f is L -Lipschitz continuous, with respect to the Euclidean norm and with $L > 0$. This implies the Descent Lemma condition described as follows:

$$f(y) \leq f(x) + \nabla f(x)^T \cdot (y - x) + \frac{L}{2} \|y - x\|^2, \forall x, y \in \text{dom}(f)$$

In fact, it gives an quadratic upper bound (approximation from above) on $f(y)$, It is used to ensure that a gradient descent iteration does not cause the function to grow too quickly.

3. Optimal value $f^* = \inf_x f(x)$ is finite and attained at x^* . It expresses that the optimal value exists and is attainable. Therefore, the problem is well-posed. In other terms,

for this algorithm to converge to a minimum, this minimum must exist and be attainable. Otherwise, we could circle around an unattainable minimum, or descend indefinitely without convergence.

In our case, thanks to the structure of the least squares objective function in linear regression, the key assumptions for gradient descent with backtracking line search are satisfied. The function is convex, continuously differentiable with a Lipschitz-continuous gradient, and admits a finite minimum which is attained. These properties guarantee the theoretical convergence of the method.

This method also uses an adaptive step-size. In particular, to ensure progress in the minimization process, the gradient descent method can rely on an adaptive strategy rather than a fixed step-size. The key idea is to adjust the step-size t dynamically at each iteration to guarantee sufficient decrease in the objective function.

We start from an initial step-size t_{init} , the algorithm iteratively reduces the step by a **contraction factor** $\alpha \in]0, 1[$ until the **Armijo condition** is satisfied. This condition in the general case is the following:

$$f(x_k + td_k) \leq f(x_k) + ct\nabla f(x_k)^\top d_k \quad (2)$$

where d_k is a descent direction, $\nabla f(x_k)^\top d_k < 0$, and $c \in (0, 1)$ is a fixed parameter that controls the sufficient decrease requirement. The term on the right represents a minimum target value that must be reached for the descent to be considered "sufficient". This condition ensures that the function decreases proportionally to the local slope. This prevents steps that are too long, which can cause the algorithm to diverge, or steps that are too small, which unnecessarily slow down convergence.

In the case of the Gradient Descent Method, where $d_k = -\nabla f(x_k)$, then the scalar product becomes:

$$\nabla f(x_k)^\top d_k = -\|\nabla f(x_k)\|_2^2.$$

Substituting this into the general Armijo condition (2), we obtain

$$\begin{aligned} f(x_k + td_k) &\leq f(x_k) + ct\nabla f(x_k)^\top d_k, \\ \iff f(x_k - t\nabla f(x_k)) &\leq f(x_k) - ct\|\nabla f(x_k)\|_2^2, \end{aligned} \quad (3)$$

While this approach adapts the step-size dynamically and makes the method more robust, it still requires an estimate of the Lipschitz constant L to set a lower bound for the step-size. In practice, the algorithm ensures $t_k \geq \min\{t_{\text{init}}, \alpha/L\}$, where α controls the sufficient decrease threshold.

Under this setting, the convergence of the average iterate is guaranteed and satisfies:

$$f(x_k) - f^* \leq \frac{1}{k} \sum_{i=1}^k (f(x_i) - f^*) \leq \frac{1}{2kt_{\min}} \|x_0 - x^*\|_2^2 \quad (4)$$

where $t_{\min}$ is the minimal accepted step-size. If α is not too small, close to the fixed step rate (if $t \in]0, 1/L[$), namely $O(1/k)$, where k is the number of iterations. Therefore, if we run the PGM algorithm for 100 iterations, the convergence rate is approximately $O(1/100)$, i.e. 1%. In other terms, we are close of the optimal value of approximately 1%. It implies that the convergence remains relatively slow, and further improvements require many additional iterations to gain small amounts of precision.

This adaptive strategy is used in all our implementations, notably, in the Gradient Descent Method, but also in the Proximal Gradient Method, as well as ISTA and its accelerated variants FISTA. However, we also implemented the fixed step size strategy to perform some comparisons.

2.3 Proximal Gradient Method (PGM)

A function $f(x)$ is called **smooth** in optimization if the function is:

1. **Differentiable** for any point in the domain.
2. $\nabla f(x)$ is **Lipschitz-continuous**, i.e. there exists $L > 0$ such that:

$$\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\|, \forall x, y \in \text{dom}(f)$$

In other words, both conditions together imply that the function changes in a controlled/gradual way. For instance, the objective function (Least Squares function) described in the previous section is smooth and that is the fundamental reason to apply the Gradient Descent method on it.

The Proximal Gradient Method extends gradient descent to non-smooth problems. In particular, this method proves to be effective for minimization problems of the following form:

$$\min_{x \in \mathbb{R}^n} f(x) + g(x) \tag{5}$$

where $f(x)$ is a differentiable function (the smooth part) and $g(x)$ is a non-differentiable but simple function (the non-smooth part). The method extends standard gradient descent by handling the non-smooth part using a proximal operator. At each iteration, it performs two following steps:

1. (Gradient step) Apply the gradient on the smooth part (on $f(x)$).

$$x_{smooth} = x_k - t_k \cdot \nabla f(x_k) \tag{6}$$

2. (Proximal step) Apply the proximal operator on the non-smooth part (on $g(x)$).

$$x_{k+1} = \text{prox}_{t_k g}(x_{smooth}) \tag{7}$$

The intuition behind this update step is the following. At each iteration, the method takes a small step down using the gradient, and then adjusts by projecting onto a nearby point that respects the non-smooth structure of the problem. This projection is assured by the proximal operation $\text{prox}_{t_k g}(z)$ which is defined as:

$$\text{prox}_{t_k g}(z) = \arg \min_x \left\{ g(x) + \frac{1}{2t_k} \|x - z\|^2 \right\} \tag{8}$$

In other terms, it acts like a generalized projection that handles non-differentiable terms. This method is particularly useful in problems like Lasso regression, where standard gradient descent or the normal equation described in Section 2.1 (linear regression) cannot be applied due to lack of regularity.

The convergence rate of this algorithm is still $O(1/k)$, where k is the number of iterations.

2.4 Specifications of the ISTA algorithm

The third algorithm considered in this project is the Iterative Shrinkage-Thresholding Algorithm (ISTA), which is a specific variant of the Proximal Gradient Method (PGM). In particular, ISTA is a specific implementation of the Proximal Gradient Method tailored to Lasso-type problems, where the non-smooth term is the l_1 -norm and the proximal step has a closed-form solution via soft-thresholding.

More formally, ISTA can be applied to a special case of the problems of the form of equation (5), where $g(x)$ is exactly on the following form: $g : \mathbb{R}^n \rightarrow \mathbb{R} : x \rightarrow \lambda \|x\|_1$. Therefore, we can rewrite the problem as follows:

$$\min_{x \in \mathbb{R}^n} f(x) + \lambda \|x\|_1 \quad (9)$$

Due to the Lasso-type problems, the proximal operator used normally for the proximal gradient method is replaced by a soft thresholding operator. Therefore, the proximal step equation (7) from the proximal gradient method becomes the "shrinkage step" defined as follows:

$$x_{k+1} = \text{shrink}(x_{\text{smooth}}, \lambda t_k) \quad (10)$$

where the shrinkage operator is applied element-wise to each coordinate of the vector x_{smooth} and is defined as:

$$\text{shrink}(x_{\text{smooth}}, \lambda t_k) = \begin{cases} x_{\text{smooth}} - \lambda t_k \cdot \text{sign}(x_{\text{smooth}}) & \text{if } |x_{\text{smooth}}| \geq \lambda t_k \\ 0 & \text{if } |x_{\text{smooth}}| < \lambda t_k \end{cases}$$

In other words, this mechanism reduces the absolute value of each component of x_{smooth} of λt_k , and cancels it completely if it is lower than this threshold value.

The application of the shrinkage operator encourages **sparsity** by setting small coefficients to zero during the optimization process.

Due to its proximity to the PGM method, the convergence rate of this method does not change. We still have a convergence rate of $O(1/k)$, where k is the number of iterations.

2.5 Specifications of the FISTA algorithm

In this project, we consider the FISTA variant of the proximal gradient method. This other variant makes the assumption that the non-smooth part $g(x)$ is closed, **convex** and possibly non-differentiable. This variant can be seen as a generalization to any closed function $g(x)$ that is convex. Moreover, it brings a great computational advantage. In fact, the theoretical convergence rate improves from $O(1/k)$ to $O(1/k^2)$. This implies that only 10 iterations are sufficient to achieve the same level of accuracy relative to the optimal value, whereas 100 iterations were previously required.

To achieve that convergence rate, the algorithm modifies the point at which the gradient step is applied. Instead of performing the gradient descent from the current iterate x_k as in ISTA, FISTA computes a linear extrapolation point y_k using the previous two iterates:

$$y_k = x_k + \beta_k(x_k - x_{k-1}).$$

Therefore, the gradient descent step is taken from y_k , and then, the proximal operator is applied to the result:

$$x_{k+1} = \text{prox}_{t_k g}(y_k - t \nabla f(y_k)).$$

This change allows FISTA to incorporate a momentum-like term into the iteration, allowing the method to take larger steps in directions of consistent improvement. The proximal operator itself remains unchanged. The key difference lies in the point where the gradient is evaluated.

3 Design

3.1 Programming language motivations

For this continuous optimization project, we chose to implement our work in Python rather than Octave due to Python’s superior code readability and maintainability. Python offers a more expressive and modern syntax, making the implementation of mathematical models and algorithms clearer and easier to debug.

Furthermore, given that we needed to test our algorithms on multiple datasets, Python proved to be a natural choice thanks to its robust ecosystem for scientific computing. In particular, we have access to some libraries as NumPy, pandas, or scikit-learn, which are widely used for data processing and numerical optimization. In our project, NumPy was essential for efficient numerical computations, particularly for handling vectors and matrices involved in optimization algorithms. Its support for fast linear algebra operations such as matrix multiplication, dot products, and norm calculations. These operations were crucial for evaluating objective functions and gradients. On the other hand, pandas greatly facilitated data manipulation and analysis. Since we worked with multiple datasets, pandas allowed us to easily load, explore, clean, and format the data in a structured way using its powerful DataFrame structure. This combination enabled us to preprocess input data efficiently and integrate it seamlessly into our optimization routines.

To enhance interactivity and streamline our development process, we combined Python with Jupyter Notebook. This environment allowed us to integrate code, visualizations, and explanatory text in a single document, facilitating better organization, experimentation, and reproducibility of our work.

3.2 The Iterative Solver

In this section we document the algorithms for the iterative solvers.

Dataset

We used the Diabetes dataset provided by the `sklearn` library. Furthermore, this dataset comes from the article named *"LEAST ANGLE REGRESSION"*, mentioned in the technical description of this project [1]. We have 442 diabetes patients, for each we have the following 10 variables: age: age in years, sex, bmi: body mass index, bp: average blood pressure, s1: tc total serum cholesterol, s2: ldl low-density lipoproteins, s3: hdl high-density lipoproteins, s4: tch total cholesterol / HDL, s5: lgt possibly log of serum triglycerides level, s6: glu blood sugar level as well as the target variable: disease progression one year after taking the previously mentioned measurements.

Libraries

Here, we mention the libraries that we used in our implementation.

- `numpy` is used for efficient numerical operations.
- `pandas` is used for handling tabular data.
- `sklearn.datasets` provides access to standard datasets like the diabetes dataset (which we used).
- `proxop.L1Norm` provides the proximal operator for the L1 norm, essential for L1 regularization.
- `proxop.scalar.Square` provides the proximal operator for the squared L2 norm.

Stopping Criterion

All iterative optimization algorithms require a stopping criterion to determine when to terminate the process. In our project, we chose to use a **scaled relative improvement** criterion, which measures the normalized change in the objective function value between two successive iterations. This formulation ensures numerical stability, even when the function value is close to zero. Specifically, the algorithm stops when the following condition is satisfied:

$$\frac{|f(x_{k+1}) - f(x_k)|}{\max\{1, |f(x_k)|\}} < \varepsilon,$$

where $f(\cdot)$ denotes the objective function, which depends on the regularized regression model used (e.g., Lasso, Ridge, or Elastic Net), and ε is a small positive threshold that controls the convergence precision. This criterion ensures that the optimization halts once further improvements in the function value become negligible relative to its scale.

Backtracking Line Search

`backtracking_line_search(X, y, beta, grad, L, t_init, alpha, c)` finds an appropriate step size for gradient descent algorithm.

Input:

- `X`, `y`, `beta` data and current model parameters.
- `grad` gradient of the loss function.
- `L` Lipschitz constant.
- `t_init` initial step size.
- `alpha` contraction factor for the step size.
- `c` controls the sufficient decrease condition (the Armijo condition).

Starts with `t_init` and decreases it geometrically `t *= alpha` until sufficient decrease is observed, it returns `t`.

Algorithm steps:

1. Compute minimum allowed step size using `min(t_init, alpha / L)`

2. Set the direction of the update to negative gradient, which points downhill in the loss function.
3. Compute the current loss with existing parameters **beta**.
4. Main loop: Try a new step, compute the loss at the new position and check if the new loss is sufficiently smaller than the old loss (Armijo condition). If not good enough, shrink **t** by multiplying with **alpha**. Check if **alpha** is too small, set it to **t_min**.
5. Once an optimal **t** is found, return it.

Gradient Descent with Backtracking

`gradient_descent_backtracking_L(X, y, L, epsilon)` implements gradient descent with adaptive step size via backtracking.

Input:

- **X**, **y**, **L** same as previously.
- **epsilon** convergence threshold on gradient norm.

Returns **beta** values attained when `gradient norm <= epsilon`.

Algorithm steps:

1. Initialize **n**: number of samples, and **d** the number of features.
2. Initialize **beta** values as vector of ones `np.ones(d)`.
3. Set the initial step size as 1 and the iteration counter to 0.
4. Main loop: Compute the gradient of the loss function. Save the current parameters to later check for convergence `beta_prev = beta.copy()`. Use backtracking line search to find an appropriate step size **eta**. Set the next round's step size to the new step size. Perform the gradient descent update `beta = beta_prev - eta * grad`. Check for convergence by testing the scaled relative improvement.
5. Return optimal **beta**.

Iterative Soft-Thresholding Algorithm (ISTA)

`ista(X, y, lambda_reg, max_iter, tol)` solves the L1-regularized least squares problem using Iterative Soft-Thresholding Algorithm (ISTA).

Input:

- **X**, **y** same as previously.
- **lambda_reg** regularization parameter controlling the sparsity.
- **max_iter** maximum number of iterations.
- **epsilon** convergence threshold.

Returns optimized **beta** values.

Algorithm steps (in case of a fixed step size):

1. Set **step_size** to $1/L$ to ensure convergence.
2. initialize **beta**.

3. Using the `L1Norm` class from `proxop` library, provide the proximal operator for the L1 norm, i.e., soft-thresholding operator.
4. Main loop: Loop until convergence or max iterations. Compute gradient of the MSE. Save a copy of `beta`. Take a gradient step and apply soft-thresholding using `prox_g.prox`. Check for convergence using the same stopping condition. Stop if relative improvement in the objective is small.
5. Return optimized `beta`.

Fast Iterative Shrinkage-Thresholding Algorithm (FISTA)

Solves the L1-regularized least squares problem using the Fast Iterative Shrinkage-Thresholding Algorithm (FISTA).

Input:

`X`, `y`, `lambda_reg`, `max_iter`, `epsilon` same as for `ista` function.

Returns optimized beta values.

Algorithm steps:

1. Initialize variables: `beta`, `theta` extrapolated point, and `t` FISTA's momentum parameter.
2. Initialize the L1 soft thresholding operator from `proxop`.
3. Main loop: Compute the gradient at `theta`. Take the proximal gradient step `prox_g.prox(theta - step_size * gradient, step_size)`. Update the momentum term using the FISTA formula for accelerated update $(1 + \sqrt{1 + 4 * t * t}) / 2$. Use the previous two iterates to compute the new extrapolated point `beta_next + ((t - 1) / t_next) * (beta_next - beta)`. Check for convergence.
4. Return `beta`

3.3 The regularized regression models

MSE Function

`loss(beta, X, y)` computes the mean squared error between calculated objective values using old beta and beta hat values, as described in the introduction section.

Inputs:

- `beta` vector of regression coefficients, 1D NumPy array.
- `X` the model matrix, a NumPy array.
- `y` response vector, 1D NumPy array.

Returns the mean of squared errors.

l_1 - norm regularization - LASSO

To implement the LASSO regularized model, we define the function `lasso_objective(beta, X, y)`.

Inputs:

- `beta` vector of regression coefficients, 1D NumPy array.
- `X` the model matrix, a NumPy array.
- `y` response vector, 1D NumPy array.

This function computes the objective of a Lasso regression model by combining the mean squared error as described above (measuring the difference between predictions $X @ \text{beta}$ and true values y) with an L1 penalty `np.linalg.norm(beta, 1)` on the coefficients `beta` to promote sparsity in the solution.

l_2 - norm regularization - RIDGE

To implement the RIDGE regularization model, we defined the function `ridge_objective(beta, X, y, lambda_reg)`.

Inputs:

- `beta` vector of regression coefficients, 1D NumPy array.
- `X` the model matrix, a NumPy array.
- `y` response vector, 1D NumPy array.
- `lambda_reg` regularization parameter (set to 0.1).

This function computes the objective of a Ridge regression model by combining the mean squared error as before with an L2 penalty (`np.dot(beta, beta)`) on the coefficients `beta`, scaled by the regularization parameter `lambda_reg`, to prevent overfitting and encourage small coefficient values.

$l_1 - l_2$ - norm regularization - ELASTIC NET

The `elastic_net_objective(beta, X, y, lambda_reg, alpha)` has the same input parameters as the previous regularization model. However, we have a float `alpha` that comes in. This `alpha` parameter controls the balance between the two types of regularization, allowing the model to interpolate between Lasso (when `alpha = 1`) and Ridge (when `alpha = 0`) behavior. The actual value of this parameter is: 0.027. This value comes from a long phase of research by trial and error.

This function calculates the Elastic Net regression objective, which combines both Lasso (L1) and Ridge (L2) regularization terms to provide a balanced model that encourages both sparsity and coefficient shrinkage.

First, it computes again the residual vector as the difference between the predicted values and the actual target values. The mean squared error (MSE) component is then calculated as the average of the squared residuals, using the dot product `np.dot(residual, residual)` and dividing by $2n$, where n is the number of samples.

To enforce sparsity, the L1 regularization term is added, which is computed as the sum of the absolute values of the coefficients `np.sum(np.abs(beta))`, scaled by the regularization parameter `lambda_reg` and the mixing parameter `alpha`.

Additionally, the L2 regularization term is included to penalize large coefficients. In fact, it is computed as half the squared Euclidean norm of `beta` (`np.dot(beta, beta)`) scaled by `lambda_reg * (1 - alpha)`.

The final objective value returned by the function is the sum of the MSE, L1, and L2 terms.

4 Results

In this section, we present the results of the implemented algorithms when using the diabetes dataset.

Gradient descent

Fixed step size: This algorithm converged in 532 iterations.

Average running time: 0.0860598087310791s

```
Iter 531: step_size = 2.2365e-01
Iter 532: step_size = 2.2365e-01
Converged in 532 iterations.
Obj: 2860.066530667335
```

Figure 1: Gradient descent

Adaptive step size: This algorithm converged in 532 iterations.

Average running time: 0.09763908386230469s

```
Iter 531: step_size = 2.2365e-01
Iter 532: step_size = 2.2365e-01
Converged in 532 iterations.
Obj: 2860.0665306673345
```

Figure 2: Gradient descent with Backtracking

ISTA

Fixed step size: This algorithm converged in 118 iterations.

Average running time: 0.022992849349975586s

```
Iter 117: Obj = 1467.781947
Iter 118: Obj = 1467.780603
Converged in 118 iterations
Obj = 1467.780603
```

Figure 3: ISTA

Adaptive Step size: This algorithm converged in 130 iterations.

Average running time: 0.067169189453125s

```
Iter 129: Obj = 1467.784920
Iter 130: Obj = 1467.783461
Converged in 130 iterations
Obj = 1467.783461
```

Figure 4: ISTA - Adaptive

FISTA - l1 Penalty

Fixed step size: This algorithm converged in 26 iterations.

Average running time: 0.009144306182861328s

```
Iter 25: Obj = 1467.388895
Iter 26: Obj = 1467.389260
Converged in 26 iterations
Obj = 1467.388895
```

Figure 5: FISTA-l1

Adaptive step size: This algorithm converged in 39 iterations.
Average running time: 0.024118423461914062s

```
Iter 38: Obj = 1467.724520
Iter 39: Obj = 1467.724368
Converged in 39 iterations
Obj = 1467.724520
```

Figure 6: FISTA-l1 - Adaptive

Proximal Gradient - Ridge

Fixed step size: This algorithm converged in 52 iterations.
Average running time: 0.0001442432403564453s

```
Iter 51: Obj = 2569.613189
Iter 52: Obj = 2569.610681
Converged in 52 iterations
Obj = 2569.610681
```

Figure 7: Ridge

Adaptive step size: This algorithm converged in 56 iterations.
Average running time: 0.00027251243591308594s

```
Iter 55: Obj = 2569.617376
Iter 56: Obj = 2569.614896
Converged in 56 iterations
Obj = 2569.614896
```

Figure 8: Ridge - Adaptive

Proximal Gradient - Elastic Net

Fixed step size: This algorithm converges in 86 iterations.
Average running time: 0.0001895427703857422s

```
Iter 85: Obj = 2547.565945
Iter 86: Obj = 2547.568434
Converged in 86 iterations
Obj=2547.568434
```

Figure 9: Elastic Net

Adaptive step size: This algorithm converges in 29 iterations.
Average running time: 0.00015282630920410156s

```

Iter 28: Obj = 2547.007217
Iter 29: Obj = 2547.008868
Converged in 29 iterations
Obj=2547.008868

```

Figure 10: Elastic Net - Adaptive

FISTA - Elastic Net

Fixed step size: This algorithm converges in 38 iterations.

Average running time: 0.0001609325408935547s

```

Iter 37: Obj = 2547.515143
Iter 38: Obj = 2547.514184
Converged in 38 iterations
Obj=2547.515143

```

Figure 11: FISTA - Elastic Net

Adaptive step size: This algorithm converges in 40 iterations.

Average running time: 0.0001659393310546875s

```

Iter 39: Obj = 2547.492848
Iter 40: Obj = 2547.491200
Converged in 40 iterations
Obj=2547.492848

```

Figure 12: FISTA - Elastic Net - Adaptive

4.1 Comparison of the results

The following table summarizes the performance of the implemented optimization algorithms when applied to different regularized regression models (RRM). The tested models include standard MSE loss, l_1 -regularization (Lasso), l_2 -regularization (Ridge), and combined $l_1 - l_2$ -regularization (Elastic Net). Each algorithm was evaluated using both a fixed and an adaptive search step size (SSS). The table reports the computation time, final objective value, and number of iterations required for convergence.

Algorithms	RRM	SSS	Avg Computation time	Obj. value	# Iterations
GDM	MSE loss	Fixed	0.0860598087310791s	2860.066530667335	532
GDM	MSE loss	Adaptive	0.09763908386230469s	2860.0665306673345	532
ISTA	L1-norm	Fixed	0.022992849349975586s	1467.780603	118
ISTA	L1-norm	Adaptive	0.067169189453125s	1467.783461	130
PGM	L2-norm	Fixed	0.0001442432403564453s	2569.610681	52
PGM	L2-norm	Adaptive	0.00027251243591308594s	2569.614896	56
PGM	L1-L2-norm	Fixed	0.0001895427703857422s	2547.568343	86
PGM	L1-L2-norm	Adaptive	0.00015282630920410156s	2547.008868	29
FISTA	L1-norm	Fixed	0.009144306182861328s	1467.388895	26
FISTA	L1-norm	Adaptive	0.024118423461914062s	1467.724520	39
FISTA	L1-L2-norm	Fixed	0.0001609325408935547s	2547.515143	38
FISTA	L1-L2-norm	Adaptive	0.0001659393310546875s	2547.492848	40

To compare these results, we answer the following three questions:

4.1.1 Which is the best search step size method ?

We can observe that both search step sizes have almost the same values in terms of number of iterations, computation time, and objective values. Their difference is really negligible.

This can be explained very easily. There is only one factor responsible for this minor difference: **alpha** factor of the backtracking line search method (Inexact Line Search). In particular, its default value given as a parameter is 0.9. To recall, the algorithm ensures the step size $t_k \geq \min\{t_{init}, \alpha/L\}$, where α controls the sufficient decrease threshold. However, in this case α -value is too close to 1. Therefore, we fall back on the fixed step size $t_k \in]0, 1/L]$. Which also explains the same convergence rate of $O(1/K)$ for the GDM, PGM and ISTA algorithms.

4.1.2 Which is the best iterative algorithm ?

Contrary to the previous point, we can clearly observe in our results an iterative algorithm called FISTA that performs much better than the others. This is consistent with our theoretical observations and the convergence rate of this algorithm of $O(1/k^2)$.

When comparing ISTA and FISTA under the same configuration, i.e. l_1 -regularization (Lasso) with an adaptive step size, we observe a significant improvement in convergence speed. ISTA requires 130 iterations to converge, whereas FISTA reaches a comparable objective value in only 39 iterations.

We can compute the improvement rate by applying the following computation:

$$\frac{130 - 39}{130} \cdot 100 = 70\%$$

Therefore, we have approximately 70% fewer iterations.

4.1.3 Which regularized regression model best fits our dataset ?

Among all the regularized regression models evaluated, the Lasso model provides the best fit for our dataset. This is evidenced by its lower objective value compared to the Ridge and Elastic Net models. The sparsity-inducing nature of the penalty allows the Lasso model to select the most relevant features while minimizing the prediction error, which results in the lowest overall objective value in our experiments.

Although both the MSE loss regression and the Ridge model achieve reasonable objective values, they do not perform as well as the Lasso model in our experiments. The main reason is that neither MSE nor Ridge promotes sparsity in the coefficient vector. As a result, they may overfit by assigning non-zero weights to irrelevant features, especially in high-dimensional settings.

In contrast, the penalty in Lasso effectively drives unnecessary coefficients to zero, leading to a more compact and interpretable model. This sparsity not only reduces complexity but also improves generalization performance, which is reflected in the lower objective value achieved by the Lasso model. This property fits well with our used dataset.

5 Conclusion

This project has demonstrated the practical implementation and comparative evaluation of several continuous optimization algorithms applied to regularized linear regression problems.

Through a modular and well-structured solver, we explored classical and modern approaches, including the Gradient Descent Method, the Proximal Gradient Method, ISTA, and FISTA.

Our experiments showed that FISTA consistently outperforms other iterative methods in terms of convergence speed, confirming its theoretical convergence rate of $O(1/k^2)$. Moreover, among all the regularization strategies tested, Lasso regression l_1 -norm provided the best fit for our dataset, achieving the lowest objective value due to its sparsity-inducing property.

The project also highlighted the importance of adaptive step-size selection and the trade-offs between computational efficiency and solution quality.

6 Project GitHub repository

You can find the code in the notebook called `iterative_solvers.ipynb`.

<https://github.com/esilberw/ContinuousOptimisation-F524>

References

- [1] BRADLEY EFRON TREVOR HASTIE IAIN JOHNSTON ROBERT TIBSHIRANI. “LEAST ANGLE REGRESSION”. In: *The Annals of Statistics 2004, Vol. 32, No. 2*, 407–499 © Institute of Mathematical Statistics (2004). DOI: <https://doi.org/10.1214/009053604000000067>.